

Soal Latihan

Modul 2: Exhaustive Search

Mata Kuliah : IBDA2011 Analisis dan Perancangan Algoritma
Dosen Pengampu : Yozef Tjandra

Soal 1

[Selection Sort]

Diberikan algoritma `selection_sort(A, N)` berikut yang mengurutkan larik A berukuran N secara *ascending*. Fungsi bantu `min_index(A[j:N])` mengembalikan indeks (dalam A) dari elemen terkecil pada rentang $A[j:N]$.

```
1 selection_sort(A, N):  
2 for j = 0, 1, ..., N-1:  
3     k = min_index(A[j:N])  
4     swap(j, k)
```

Tugas: Modifikasilah pseudocode di atas menjadi `selection_sort_desc(A, N)` sehingga A terurut secara *descending*. Anda boleh mengasumsikan tersedianya fungsi analog `max_index(A[j:N])` (definisi serupa `min_index`, tetapi mengembalikan indeks elemen terbesar). Tunjukkan bagian mana saja yang perlu diubah.

Soal 2

[Selection Sort]

Domain: $N \geq 2$, seluruh elemen A berbeda (distinct).

Perhatikan algoritma berikut:

```
1 selection_sort_desc(A, N):  
2 for j = N-1 downto 0:  
3     idx = 0  
4     for i = 1 to j-1:  
5         if A[i] < A[idx]:  
6             idx = i  
7     swap(j, idx)
```

Tugas:

- Algoritma di atas **tidak selalu** menghasilkan A yang terurut *descending* dengan benar. Berikan sebuah instance A dan N di mana algoritma ini gagal, disertai jejak eksekusi yang menunjukkan baris mana yang bermasalah dan mengapa.
- Perbaiki bug tersebut. Perbaikan Anda seharusnya hanya memerlukan perubahan pada **satu baris kode saja**; bagian lain sudah benar dan tidak perlu diubah.

Soal 3

[Brute-Force String Matching]

Domain: $n \geq m \geq 1$; T dan P atas alfabet yang sama dengan minimal 2 simbol berbeda.

Diberikan algoritma *brute-force string matching* berikut, yang mengembalikan indeks kemunculan **pertama** pattern P (berukuran m) di dalam teks T (berukuran n), atau -1 jika tidak ditemukan:

```
1 BruteForceStringMatch(T, n, P, m):  
2 for i ← 0 to n-m:  
3     j ← 0  
4     while j < m and P[j] = T[i+j]:  
5         j ← j+1
```

```

6     if j = m:
7         return i
8     return -1

```

Tugas:

- Modifikasi algoritma di atas agar mengembalikan indeks kemunculan **terakhir** pattern P di dalam T , atau -1 jika tidak ditemukan sama sekali. **Ilustrasi presisi “kemunculan terakhir”**. Misalkan $T = \text{ABCABCABC}$ ($n = 9$) dan $P = \text{ABC}$ ($m = 3$). Pattern ini muncul pada indeks awal $i = 0$, $i = 3$, dan $i = 6$. Yang dimaksud “kemunculan terakhir” adalah **indeks awal terbesar** dari kemunculan tersebut, yaitu $i = 6$ — bukan indeks karakter terakhir teks ($i = 8$), dan bukan pula posisi akhir dari jendela kemunculan tersebut.
- Tentukan karakteristik instance kasus **terbaik** untuk algoritma hasil modifikasi Anda, dan buktikan kelas efisiensinya dengan menurunkan $T(n)$ menggunakan notasi sigma.
- Tentukan karakteristik instance kasus **terburuk**, dan buktikan kelas efisiensinya dengan cara yang sama.

Soal 4

[Exhaustive Search: Counting Substrings]

Perhatikan permasalahan pencacahan berikut. Pada suatu teks yang diberikan, cacahlah banyaknya substring yang **diawali** dengan karakter A dan **diakhiri** dengan karakter B. Sebagai contoh, terdapat empat substring demikian pada teks CABAAXBYA.

Domain: $n \geq 0$; T adalah string sepanjang n atas suatu alfabet yang memuat sekurang-kurangnya karakter A dan B.

Tugas:

- Rancang algoritma *brute-force* (berbasis *exhaustive search* atas seluruh pasangan posisi) untuk masalah ini, dengan kompleksitas waktu kasus terburuk berorde **maksimum kuadratik**, $O(n^2)$.
- Tentukan karakteristik instance kasus terbaik dan kasus terburuk untuk algoritma Anda.
- Buktikan kelas efisiensi masing-masing kasus menggunakan notasi sigma.
- [Tantangan] Rancang algoritma untuk masalah yang sama dengan kompleksitas waktu kasus terburuk **lebih baik dari kuadratik**. Buktikan kelas efisiensinya, dan bandingkan dengan hasil (c).

Soal 5

[Exhaustive Search Adhoc]

Tugas:

- Rancang sebuah algoritma *exhaustive search* untuk menghitung nilai polinomial

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

pada suatu titik x_0 .

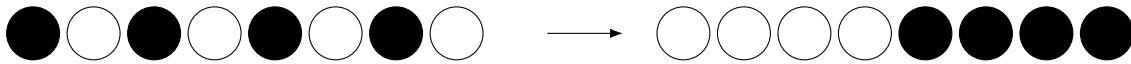
- Analisislah efisiensi **waktu** dan **ruang** dari algoritma Anda.
- [Tantangan] Bisakah dirancang algoritma yang **subkuadratik** (lebih cepat dari kuadratik) untuk masalah ini?

Soal 6

[Exhaustive Search Adhoc]

Cakram berselang-seling. Anda memiliki satu baris berisi $2n$ cakram dengan dua warna, n gelap dan n terang, tersusun berselang-seling: gelap, terang, gelap, terang, dan seterusnya. Anda ingin

memindahkan seluruh cakram gelap ke ujung kanan, dan seluruh cakram terang ke ujung kiri. Satu-satunya langkah yang diperbolehkan adalah **menukar posisi dua cakram bertetangga**.



Tugas:

- Rancang sebuah algoritma untuk menyelesaikan teka-teki ini, dan tentukan banyaknya langkah (penukaran) yang dibutuhkan.
- Tentukan efisiensi **waktu** dan **ruang** dari algoritma yang Anda rancang, nyatakan dalam notasi Big-Theta, serta buktikan.

Soal 7

[Brute Force Convex Hull]

Diberikan m garis $\ell_i : a_i x + b_i y + c_i = 0$ (untuk semua $i = 1, \dots, m$) dan k titik $Q_j = (x_j, y_j)$ (untuk semua $j = 1, \dots, k$).

Asumsi: tidak ada dua garis kandidat yang sejajar berimpit (tidak ada solusi ambigu akibat dua sisi berimpit); tidak ada titik Q_j yang tepat berada di atas garis ℓ_i manapun (posisi umum/*general position*), sehingga uji tanda pada tiap titik selalu tegas positif atau negatif, tidak pernah nol.

Tugas: Rancang algoritma *brute-force* untuk menentukan (jawaban **Ya/Tidak** saja) apakah terdapat kumpulan segmen dari garis-garis tersebut yang membentuk sebuah poligon convex yang melingkupi seluruh k titik di dalamnya. Analisislah efisiensi waktu algoritma Anda.

Soal 8

[Brute Force Convex Hull]

Domain: $n \geq 3$ titik, tidak ada dua titik yang duplikat (berimpit); **tidak** diasumsikan *general position* — domain ini sengaja memuat kasus-kasus kolinear yang akan diuji pada bagian (a)–(c).

Kode dasar

```

1 BruteForceConvexHull(P, n):
2   for i ← 0 to n-1:
3     for j ← 0 to n-1, j ≠ i:
4       sisiKiri ← true
5       sisiKanan ← true
6       for k ← 0 to n-1, k ≠ i, k ≠ j:
7         d ← cross(P[j]-P[i], P[k]-P[i])
8         if d < 0: sisiKiri ← false
9         if d > 0: sisiKanan ← false
10      if sisiKiri or sisiKanan:
11        output segmen (P[i], P[j]) sebagai sisi hull

```

Tugas: Tunjukkan kegagalan algoritma pada kasus (a)–(c) berikut (berikan suatu contoh instance yang menggagalkan algoritma dan jelaskan bagaimana persisnya algoritma akan mengoutputkan hasil yang salah), lalu rancang ulang agar mengakomodasi ketiganya.

- Ketika ada persis 3 titik kolinear di antaranya sehingga garis kolinear tersebut merupakan bagian dari convex hull.
- Ketika semua titik berada di boundary sebuah persegi panjang dan jawaban dari convex hull yang benar haruslah berupa persegi panjang itu sendiri.
- Ketika semua titik berada pada sebuah garis saja (jawaban yang benar seharusnya convex hull berupa segmen garis terpendek yang memuat semua titik).
- Perbaiki kode tersebut agar bisa mengakomodasi semua kasus ekstrem di atas. Pertimbangkan untuk merombak total kode jika diperlukan, ketimbang memberikan “tambalan”.

Soal 9

[Traversal Tree]

Modifikasi Masalah Harta Karun 1D: larik A berukuran $N = 5$ (indeks 0 sampai 4).

Aturan pergerakan **diubah** menjadi: dari posisi i , Anda dapat berpindah

- **1 langkah** ($i \rightarrow i + 1$), atau
- **3 langkah** ($i \rightarrow i + 3$) ke kanan.

Perjalanan dimulai dari indeks 0 dan berakhir (“out”) begitu posisi mencapai atau melewati $N = 5$.

Tugas: Buatlah **pohon penelusuran (traversal tree)** lengkap untuk kasus ini — belum perlu meninjau nilai harta karun $A[i]$, cukup struktur traversal-nya saja.

Soal 10

[Traversal Tree]

Modifikasi Masalah Harta Karun menjadi versi **2D**: penjelajahan dimulai dari sel $(0,0)$ pada suatu grid tak terbatas.

Pada setiap langkah, tersedia **4 pilihan gerak**, masing-masing tepat 1 sel:

- **atas** $(x, y) \rightarrow (x, y+1)$,
- **bawah** $(x, y) \rightarrow (x, y-1)$,
- **kiri** $(x, y) \rightarrow (x-1, y)$, atau
- **kanan** $(x, y) \rightarrow (x+1, y)$.

Tugas: Buatlah pohon penelusuran hingga **maksimum 2 langkah** dari $(0,0)$ (belum perlu meninjau nilai apapun di tiap sel — murni struktur traversal-nya).

Soal 11

[Exhaustive Search Optimisation]

Diberikan grid harta karun 2D. Mulai dari sel $(0,0)$, tersedia 4 arah gerak (masing-masing 1 sel): atas, bawah, kiri, kanan. **Tepat 2 langkah** wajib dilakukan (tidak ada opsi berhenti lebih awal). Total harta = jumlah nilai seluruh sel yang disinggahi, **termasuk** $(0,0)$ di awal.

Nilai harta karun tiap sel yang relevan:

$y \backslash x$	-2	-1	0	1	2
2		2	4	3	
1	1	2	5	3	2
0	-4	-2	2 (start)	1	1
-1	-6	-5	-3	-2	-2
-2		-6	-5	-4	

Tugas: Hitunglah total harta karun **maksimum** yang dapat diperoleh dalam tepat 2 langkah dari $(0,0)$.

Soal 12

[Exhaustive Search Optimisation]

Diberikan larik harta karun 1D A berukuran $N = 5$:

i	0	1	2	3	4
$A[i]$	3	-5	4	2	-6

Aturan sama seperti Soal 9: mulai dari indeks 0, tiap langkah boleh maju 1 atau 3 posisi, berhenti otomatis begitu posisi ≥ 5 . Total harta = jumlah $A[i]$ pada **seluruh indeks yang benar-benar disinggahi** (indeks yang dilompati tidak dihitung).

Tugas: Hitunglah total harta karun **maksimum** yang dapat diperoleh. Tunjukkan seluruh kandidat solusi (gunakan traversal tree dari Soal 9 sebagai kerangka) beserta nilainya masing-masing.

Soal 13

[DFS/BFS]

Cerita. Sebuah mesin pencetak kode biner mencetak suatu kode target S (string biner panjang n) secara bertahap dalam beberapa **tahap** (*batch*). Mesin mulai dari string kosong. Pada setiap tahap, mesin menghitung m = banyaknya bit 1 pada bagian yang **sudah tercetak sejauh ini**, lalu boleh mencetak (meng-*append*) sebanyak k karakter berikutnya dari S , dengan $1 \leq k \leq m + 1$ (dipilih **bebas**, asalkan tidak melebihi sisa panjang S). Tujuan: mencetak seluruh S dalam **jumlah tahap paling sedikit**.

Contoh 1. $S = 101$ ($n = 3$).

- Posisi $L = 0$: belum ada yang tercetak, $m = 0 \Rightarrow$ maksimum $k = 1$. **Wajib** mencetak 1 karakter: $S[0] = 1$. Posisi jadi $L = 1$.
- Posisi $L = 1$ (tercetak “1”, $m = 1$): maksimum $k = 2$. Dua pilihan:
 - $k = 1$: cetak $S[1] = 0 \rightarrow L = 2$. Dari sini $m = 1$, cetak $S[2] = 1 \rightarrow L = 3$. **Total 3 tahap.**
 - $k = 2$: cetak $S[1..2] = 01$ sekaligus $\rightarrow L = 3$ (selesai). **Total 2 tahap.**
- **Minimum = 2 tahap**, dicapai lewat rute langsung.

Contoh 2. $S = 000000$ ($n = 6$). Karena tidak pernah ada bit 1 yang tercetak, $m = 0$ selamanya, sehingga maksimum $k = 1$ di setiap tahap. Mesin terpaksa mencetak satu karakter per tahap. **Minimum = 6 tahap** (satu-satunya kemungkinan).

Domain: S string biner panjang $n \geq 1$ atas alfabet $\{0, 1\}$, isi S diketahui/tetap di awal.

Kode DFS rekursif yang diberikan (mengeksplorasi seluruh kemungkinan dekomposisi, mencatat jumlah tahap minimum ke variabel global `minSteps`):

```

1 BuildBitstring(S, n):
2     minSteps ← infinity
3     DFSExplore(S, n, 0, 0)
4     return minSteps

```

```

1 DFSExplore(S, n, L, langkah):
2     if L = n:
3         if langkah < minSteps:
4             minSteps ← langkah
5             return
6     m ← CountOnes(S, 0, L-1)
7     for k ← 1 to m+1:
8         if L + k ≤ n:
9             DFSExplore(S, n, L+k, langkah+1)

```

Tugas:

- Apa yang dilakukan baris 8 dari kode di atas? Apa efeknya jika kode tersebut dihapus?
- Untuk $S = 10011$ ($n = 5$), gambarkan **seluruh pohon rekursi** yang dieksplorasi `DFSExplore`, dan tentukan jumlah tahap minimum beserta rute yang mencapainya.
- Tulis ulang algoritma di atas menjadi versi **BFS memakai queue eksplisit**, yang **berhenti seketika** saat pertama kali mencapai $L = n$. *Petunjuk:* Gunakan array/tabel `visited` agar posisi L yang sama tidak diproses berulang.
- Jalankan (telusuri) algoritma BFS Anda pada $S = 10011$, dan tunjukkan bahwa hasilnya konsisten dengan (a).

Input/Output untuk verifikasi kode:

- Input: $S = 10011 \rightarrow$ Output: minimum = 3 tahap (contoh rute: $0 \rightarrow 1 \rightarrow 3 \rightarrow 5$).
- Input: $S = 11111 \rightarrow$ Output: minimum = 3 tahap.

Soal 14

[DFS/BFS]

Cerita. Seekor tikus berada di sel $(0,0)$ dalam labirin berpetak 2D berukuran 4×4 (koordinat $x, y \in \{0, 1, 2, 3\}$) dan ingin mencapai keju di sel $(2,2)$. Beberapa sel berupa **tembok** (tidak bisa dilewati): $(1,0), (1,1), (1,2)$.

		keju	
start			

Domain: grid 4×4 , 4 arah gerak (atas/bawah/kiri/kanan, masing-masing 1 sel), gerakan ke luar grid atau ke sel tembok tidak diperbolehkan.

Kode BFS (queue) yang diberikan, menemukan panjang lintasan terpendek:

```
1 BFS_Shortest(start, goal, isWall):
2     visited ← {}
3     Q ← empty queue
4     push(Q, (start, 0))
5     visited ← visited U {start}
6     while not Q.empty():
7         (pos, jarak) ← pop(Q)
8         if pos = goal:
9             return jarak
10        for arah in {atas, bawah, kiri, kanan}:
11            pos' ← pos + arah
12            if pos' ≠ tembok and pos' is not out-of-bound and pos' not in visited:
13                visited ← visited U {pos'}
14                push(Q, (pos', jarak+1))
15    return -1
```

Tugas:

- Jalankan versi BFS pada labirin di atas dan laporkan panjang lintasan terpendeknya.
- Tulis ulang algoritma di atas menjadi **DFS rekursif** (tetap gunakan himpunan **visited** agar tidak terjebak siklus tak berhingga), yang mengembalikan panjang lintasan **pertama** yang ditemukan (bukan lintasan terpendek).
- Jalankan versi DFS Anda dengan **urutan prioritas arah**: kanan, atas, kiri, bawah. Tunjukkan bahwa DFS **tidak** menemukan lintasan sependek BFS, dan jelaskan **mengapa** secara struktural BFS menjamin lintasan terpendek sedangkan DFS tidak.

Input/Output:

- Input: grid 4×4 di atas, start $(0,0)$, goal $(2,2) \rightarrow$ Output BFS: **6 langkah**.
- Output DFS (prioritas kanan $\rightarrow$ atas $\rightarrow$ kiri $\rightarrow$ bawah): **8 langkah** (lebih panjang).

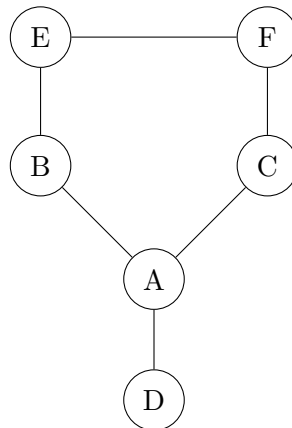
Soal 15

[DFS/BFS]

Cerita. Sebuah maskapai memiliki rute penerbangan antar 6 kota — A, B, C, D, E, F — direpresentasikan sebagai graf **tak berarah**.

Graf (adjacency list, tetangga terurut alfabetis):

A: [B, C]
B: [A, E]
C: [A, F]
D: [A]
E: [B, F]
F: [C, E]



Domain: graf tak berarah; kota yang dicek adalah yang terjangkau dari A.

Kode BFS (queue) yang diberikan (mencetak urutan kunjungan, tetangga diproses terurut alfabetis):

```
1 CheckConnectivity_BFS(adj, start):  
2     visited ← {start}  
3     urutan ← [start]  
4     Q ← empty queue  
5     push(Q, start)  
6     while not Q.empty():  
7         kota ← pop(Q)  
8         for tetangga in adj[kota] (terurut alfabetis):  
9             if tetangga not in visited:  
10                visited ← visited U {tetangga}  
11                urutan ← urutan + [tetangga]  
12                push(Q, tetangga)  
13     return urutan
```

Tugas:

- Tulis ulang algoritma di atas menjadi **DFS rekursif** (tetap mencetak urutan kunjungan, tetangga diproses terurut alfabetis).
- Jalankan **kedua versi** (BFS dan DFS) pada graf di atas mulai dari A, dan laporkan urutan kunjungan masing-masing.

Input/Output:

- Input: graf di atas, start = A → Output BFS: A, B, C, D, E, F.
- Output DFS (versi Anda): A, B, E, F, C, D.

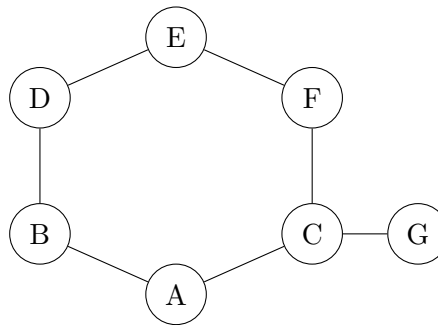
Soal 16

[DFS/BFS]

Cerita. Sebuah kabar disebarakan dalam jaringan pertemanan (graf tak berarah) mulai dari Andi (A).

Graf (adjacency list, tetangga terurut alfabetis):

A: [B, C]
B: [A, D]
C: [A, F, G]
D: [B, E]
E: [D, F]
F: [C, E]
G: [C]



Kode DFS rekursif yang diberikan, mencoba menghitung “jarak” dari sumber A dengan cara naif: mencatat **kedalaman rekursi** saat suatu kota pertama kali dikunjungi sebagai jaraknya dari A.

```
1 DFS_NaiveDepth(adj, kota, kedalaman, depth):  
2     if kota in depth:  
3         return  
4     depth[kota] ← kedalaman  
5     for tetangga di adj[kota] (terurut alfabetis):  
6         DFS_NaiveDepth(adj, tetangga, kedalaman+1, depth)
```

(dipanggil awal via `DFS_NaiveDepth(adj, A, 0, {})`)

Tugas:

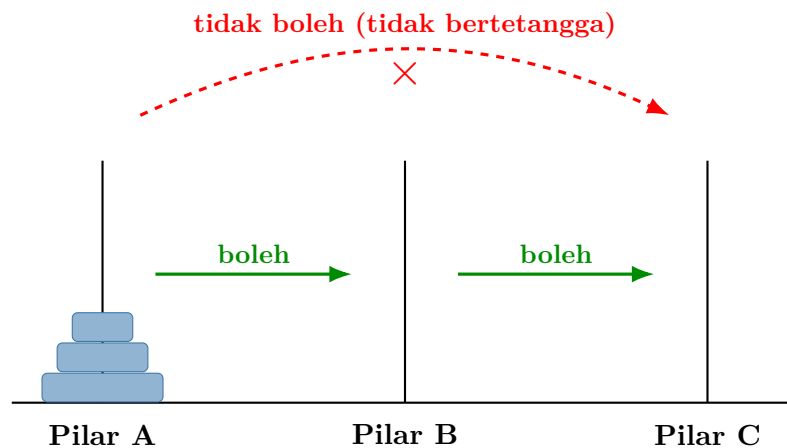
- Jalankan algoritma DFS di atas pada graf tersebut mulai dari A (urutan tetangga alfabetis), dan laporkan nilai **depth** yang dihasilkan untuk setiap kota. Berdasarkan hasil ini, jelaskan mengapa pendekatan “kedalaman rekursi sebagai jarak” ini **tidak dapat diandalkan** secara umum untuk menghitung jarak (dan karenanya eksentrisitas) pada graf yang memiliki siklus.
- Rancang versi **BFS** (menggunakan queue) untuk menghitung jarak sesungguhnya (jumlah hop minimum) dari A ke setiap kota, dan tentukan **eksentrisitas** A (jarak maksimum dari A ke node manapun yang terjangkau).
- Jelaskan mengapa pendekatan BFS Anda pada (b) **selalu benar**, secara struktural, dalam menghitung jarak pada graf apa pun — termasuk graf bersiklus seperti pada soal ini.

Soal 17

[Analisis BFS/DFS]

Cerita — Menara Hanoi Lorong Sempit. Sebuah kuil kuno memiliki tiga pilar batu tersusun berjejer dalam satu lorong sempit: Pilar A — Pilar B — Pilar C. Karena struktur lorong yang sempit, cakram **hanya boleh dipindahkan antar pilar yang bertetangga secara fisik** ($A \leftrightarrow B$

atau $B \leftrightarrow C$) — **tidak boleh** langsung antara A dan C. Para biksu harus memindahkan n cakram berukuran berbeda dari Pilar A ke Pilar C, mengikuti aturan Hanoi standar lainnya.



Domain: $n \geq 0$ cakram, ukuran seluruhnya berbeda. Basic operation = satu perpindahan satu cakram.

Kode DFS rekursif:

```

1 HanoiLinear(n, src, mid, dst):
2     if n = 0:
3         return
4     HanoiLinear(n-1, src, mid, dst)
5     pindahkan cakram-n dari src ke mid
6     HanoiLinear(n-1, dst, mid, src)
7     pindahkan cakram-n dari mid ke dst
8     HanoiLinear(n-1, src, mid, dst)

```

(dipanggil awal via $\text{HanoiLinear}(n, A, B, C)$)

Tugas:

- Verifikasi untuk $n = 2$: tunjukkan bahwa jejak eksekusi menghasilkan 8 langkah perpindahan total, dengan hasil akhir seluruh cakram berpindah dengan benar ke Pilar C.
- Diberikan relasi rekurensi $T(n) = 3T(n-1) + 2$, $T(0) = 0$, di mana $T(n)$ = banyaknya perpindahan cakram untuk n cakram. **Buktikan** relasi ini benar-benar merepresentasikan algoritma di atas, dengan menganalisis baris kode secara langsung.
- Diberikan formula eksplisit $T(n) = 3^n - 1$. **Buktikan** formula ini (gunakan induksi **atau** forward substitution/generalisasi).
- Tentukan kelas efisiensi algoritma ini.

Soal 18

[Analisis BFS/DFS]

Cerita — Ruang Harta Karun Berportal. Ruang harta karun tersusun 1D, ruang 0 sampai n . Dari ruang i , tersedia gerbang teleportasi: **satu gerbang pendek** (berpindah tepat 1 ruang ke kanan, hanya ada 1 cara), dan **dua gerbang panjang berbeda warna** (Biru dan Merah) yang **keduanya** berpindah tepat 2 ruang ke kanan (dihitung sebagai **dua rute berbeda**). Yang ingin dihitung: **banyaknya rute berbeda** untuk mencapai ruang n dari ruang 0.

Domain: $n \geq 0$ integer.

Kode BFS (queue):

```

1 CountRoutes(n):
2     total ← 0

```

```

3  Q ← queue kosong
4  push(Q, 0)
5  while Q tidak kosong:
6      i ← pop(Q)
7      if i = n:
8          total ← total + 1
9      else:
10         if i+1 ≤ n:
11             push(Q, i+1)          // gerbang pendek
12         if i+2 ≤ n:
13             push(Q, i+2)          // gerbang Biru
14             push(Q, i+2)          // gerbang Merah (rute berbeda meski tujuan sama)
15 return total

```

Tugas:

- Verifikasilah untuk $n = 3$:** tunjukkan bahwa algoritma di atas mengembalikan 5 rute.
- Diberikan relasi rekurensi $L(n) = L(n-1) + 2L(n-2)$ untuk $n \geq 2$, dengan $L(0) = 1, L(1) = 1$. **Buktikan** relasi ini merepresentasikan algoritma di atas dengan benar.
- Diberikan formula eksplisit $L(n) = \frac{2^{n+1} + (-1)^n}{3}$. **Buktikan** formula ini (gunakan induksi **atau** persamaan karakteristik).
- Tentukan efisiensi **waktu** dan **ruang** dari algoritma ini.

Soal 19

[Analisis BFS/DFS]

Diberikan kembali algoritma `CheckConnectivity_BFS` dari Soal 15 (kode BFS yang diberikan pada soal tersebut, sebelum dikonversi menjadi DFS).

Tugas:

- Tentukan basic operation yang perlu dicacah untuk mengevaluasi efisiensi algoritma tersebut.
- Tentukan karakteristik instance kasus **terbaik** dan **terburuk**, beserta kelas efisiensi masing-masing, dan buktikan.
- Analisis efisiensi **ruang** algoritma ini (nyatakan total = memori input + auxiliary).

Soal 20

[Analisis BFS/DFS]

Diberikan kembali algoritma `DFS_NaiveDepth` dari Soal 16 (kode DFS yang diberikan pada soal tersebut, sebelum diminta merancang versi BFS).

Tugas:

- Tentukan basic operation yang perlu dicacah untuk mengevaluasi efisiensi algoritma tersebut.
- Tentukan karakteristik instance kasus **terbaik** dan **terburuk**, beserta kelas efisiensi masing-masing, dan buktikan.
- Analisis efisiensi **ruang** algoritma ini (nyatakan total = memori input + auxiliary).